

UWE-4 ML Models

Variational Autoencoder & Baseline

Phase 5: Model Selection History & Current Baseline

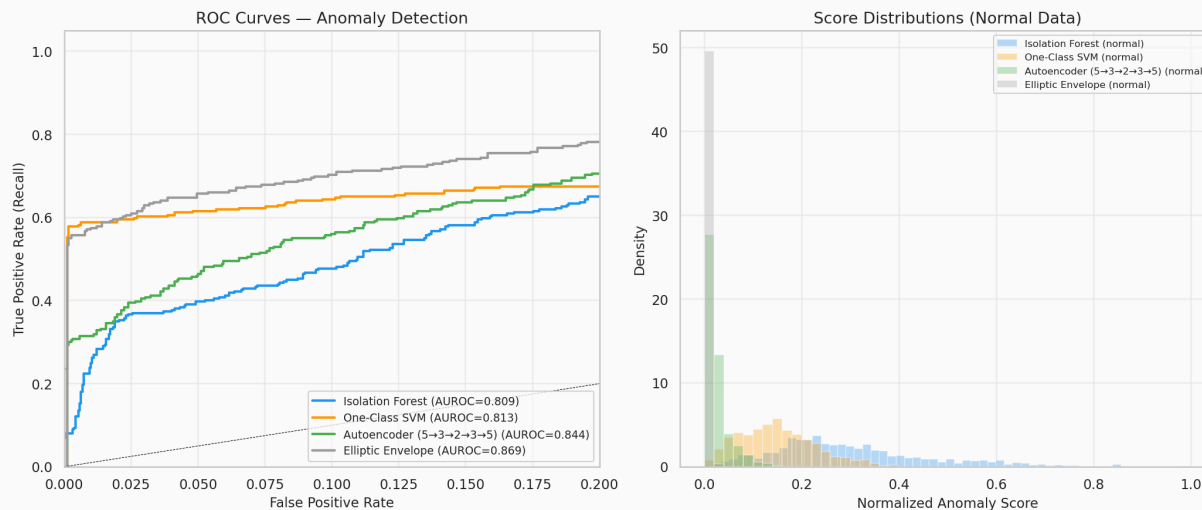
Benchmarking Without Failures

Because **real** spacecraft anomalies are undocumented in our clean set, we benchmark offline models using synthetic fault injection on a held-out test set (20%).

- **1. Panel Failure:** Force `batt_current` to discharge while `temp_panel_z` shows sunlight. (Severed array)
- **2. Thermal Runaway:** Inject a large positive step into battery temperatures. (Internal short)
- **3. Sensor Stuck:** Historical notebook-only experiment, not part of the current shipped benchmark script.

Competitive AI Models

We tested 4 unsupervised mathematical models against the synthetic faults.

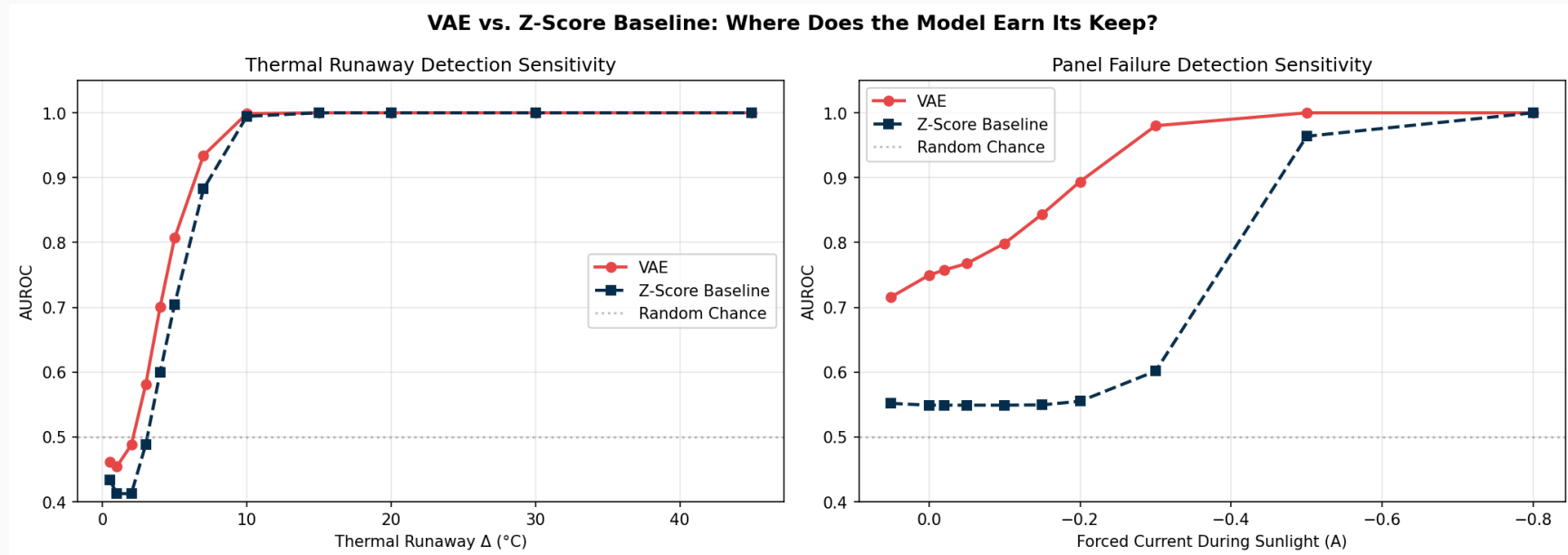


Historical outcome:

- **Elliptic Envelope:** useful exploratory baseline, but not retained in the current repository path.
- **PyTorch VAE:** current repository baseline for training and offline synthetic-fault benchmarking.

The “Honest” Benchmark: Sensitivity Sweep

A 100% detection rate on extreme faults proves nothing; basic thresholds can catch +45°C thermal spikes. We swept the fault magnitudes from subtle to extreme to find the operational crossover where the VAE’s multivariate awareness actually outperforms a simple Z-Score limit.



The “Honest” Benchmark: Sensitivity Sweep

The Verdict: The VAE massively outperforms dumb thresholding (Z-Score) during subtle anomalies (like a 0.1A current drop during sunlight), while performing equally well on obvious, extreme faults.

Stage 1: Overall Score

Goal: The current codebase uses a VAE-only pipeline.

Logic: Run the VAE and compute MSE + KLD per frame. The operating threshold is calibrated during training and persisted in the model metadata artifact.

Result: Provides a deployment-ready anomaly threshold for real-time inference.

Stage 2: Feature Diagnosis

Goal: We need per-feature attribution after a frame is flagged.

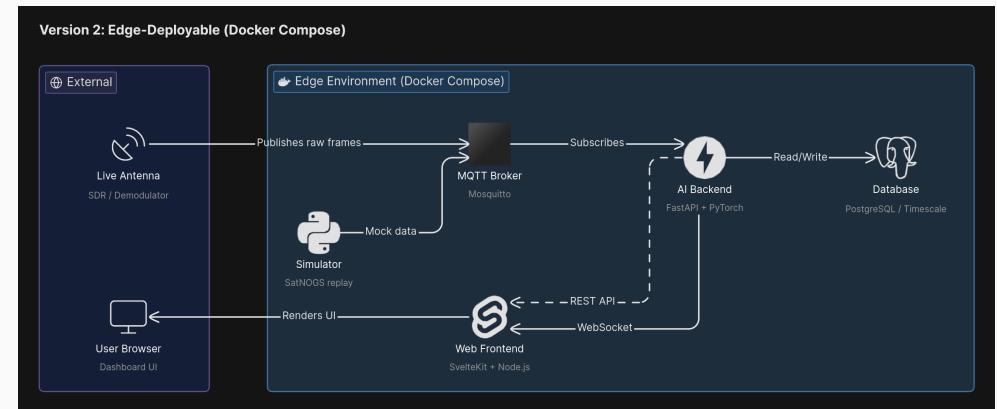
Logic: Inspect the reconstruction error feature-by-feature and identify the dominant contributor.

Result: Provides real-time subsystem attribution for both dashboards and offline benchmarking.

Architecture: The Wedjat Stack

We have deployed the real-time wedjat as a 5-container microservice stack:

- **MQTT Broker (Mosquitto):** Live telemetry ingestion.
- **Database (TimescaleDB):** Time-series telemetry storage.
- **Backend API (FastAPI):** Inference and WebSocket streaming.
- **Frontend (SvelteKit):** Real-time dashboard and ML insights.
- **Simulator:** Replays data to test the pipeline.



Conclusion: Current State

1. **Implemented now:** Offline ML pipeline, VAE training, Dockerized 5-component microservice stack, FastAPI backend, and SvelteKit real-time dashboard.
2. **Next Step:** Integrate live SDR (Software Defined Radio) ingress, expand decoder coverage, and tune production anomaly thresholds based on live operational feedback.